Reflection

Boréal Marché Customer Intelligence Platform · Manish Chauhan · 26 October 2026

The hardest problem: keeping the evaluation honest about time

The hardest part of this project was not a model or a cloud service. It was making sure that every number I reported came from a test that could not see the future. A retention model predicts what a customer will do in the next 90 days, so any feature, any rule, any threshold or any split that quietly uses information from after the cutoff produces a number that looks excellent and means nothing. In this project that risk showed up in four different places, and I only saw the later ones because I had learned to look for the first.

Head A (features and labels). My first Gold table used point-in-time features, but I did not want to trust that by argument. I ran three tests. I recomputed recency and spend from the raw rows before the 31 August 2011 cutoff, and they matched the table exactly. I injected two label-window aggregates on purpose, and PR-AUC jumped from 0.857 to 1.000, which proved that my harness would catch leakage. And I checked that the highest feature-label correlation was 0.431, well under the 0.85 trigger. **Head B (rules).** In D2 I mined rules on all 36,969 baskets. For the temporal hit-rate test that is leakage, because the rules would already have seen the same customers' later purchases. In D3 I re-mined on the 29,850 baskets before the cutoff only, which is why the D2 and D3 rule counts differ. **The held-out slice.** I froze model selection before touching it. When Random Forest scored slightly higher (0.8787 against 0.8753), I did not switch, because choosing after the test would have made it a validation set.

What worked was treating "no leakage" as a claim that needs a test that could fail. What I would still flag as imperfect: I chose the 0.82 threshold by sweeping profit on the held-out slice. When I re-checked on Monday by choosing it on the calibration slice, I got the same threshold and the same held-out profit, so the conclusion stands, but that check should have been the original design.

One thing I got wrong, and corrected

I assumed my train, calibration and held-out split was deterministic. It was not. The slices are cut by last-purchase date, and about 34 customers share the same last-purchase day at the 85% boundary. My code sorted with a default, unstable sort, so row order decided which of those customers landed in the held-out slice. The weekly retraining workflow exposed it. Its first run in GitHub Actions scored the candidate at PR-AUC 0.8782 against 0.8764 for the deployed model, an apparent gain of 0.0018 from retraining the same recipe on the same, unchanging data. The gate (a gain of at least 0.005) correctly kept the current model, but the number bothered me, because nothing had changed. Tracing it back, the "gain" was the difference between two slightly different held-out slices.

I fixed the split with a stable sort and a customer-ID tie-break, kept the 0.005 margin precisely because noise of this size exists, and rewrote the description of the job. The data set is static, so the workflow demonstrates a mechanism and does not find real improvement, and I now say that in the D4 note and the final report. I have not fully closed the loop: recomputing the held-out PR-AUC today gives 0.8726 against 0.8753 in the model metadata, and I did not investigate the remaining difference.

A smaller correction came on the last day. I had written in the executive summary that targeted offers beat the blanket coupon in every scenario I tested. That was true only when I changed one assumption at a time. When I varied coupon cost, margin and recovery rate together, the targeted policy lost in 5 of 27 combinations. I rewrote the summary, put the script in the repository, and made the recommendation a pilot with a control group, which is the honest conclusion from that result.

What I would do differently

- **Deploy a thin slice in week one.** Almost every deployment surprise arrived late: App Runner was closed to new customers, the first deploy failed at role assumption because the GitHub OIDC token subject now contains numeric owner and repository IDs (CloudTrail showed the real subject, and loosening the policy with wildcards did not help), and I found only while wiring the container for D4 that the artifact committed at tag v0.3 was an earlier model than the one in my D3 report. An empty service deployed early would have exposed all three while they were cheap.
- **Look at the weak segment when I first see it.** My D3 error analysis already showed that low-spend customers had 1.9% recall for the retained class. I called it "economically wasteful rather than unfair" and moved on. In D5 it turned out to be the central finding: the model never scores new or low-spend customers above 0.52, so all of them fall under the 0.82 threshold, and two of my four mitigations (recalibration and reweighting) could not help because the features lack the information. I would have started collecting better features, and designed the human review step around that group, in D3.
- **Get real business numbers before building the profit case.** The $5 coupon, 20% margin and 30% recovery rate are my assumptions, and the result depends on them. I would ask for the real figures at the start and run the sensitivity analysis before writing any dollar claim.
- **Write the hard evidence into the repository as scripts from the start.** Reproducing numbers late, from memory of what I ran, is how small inconsistencies (0.8726 against 0.8753, $201 against $235 for the blanket baseline) crept in.

How I worked

I used Claude (Anthropic) as a coding and writing assistant. It was useful for speed and, more than once, wrong: an explanation score that tied because of the calibrator's step function, a library version I assumed, an early D5 draft that did not follow the specification. What kept the project honest was not trusting its output or mine, and relying on tests, CI, the live service and the specification to say who was right. Section 13 of the final report lists the corrections.